Group Project Instructions

Dr. Franziska Bender, Dr. Aurélien Sallin

20.04.2026

Overview

For your final project, your team will choose a real-world economic question that genuinely interests you, identify the relevant data series on FRED (Federal Reserve Bank of St. Louis), and build a small data pipeline to analyze it end to end. The project is designed to reflect how data work is actually done in practice: collaboratively, reproducibly, and with reusable code.

You will retrieve economic time series from the FRED API, store them in a relational database, and write a Python class that reads from that database to perform your analysis. The result will be a short brief on your chosen economic topic presenting your findings, a reusable toolkit that others could apply to explore related questions, and a User Guide documenting how to use it.

The project is assessed on three dimensions: how well you collaborate using Git and GitHub, how cleanly your Python environment and repository are set up, and the quality of your programming work.

Objectives

Students should demonstrate their ability to:

- Collaborate effectively using Git and GitHub.
- Set up and manage a reproducible Python environment with `uv`.
- Retrieve data from an external API and handle credentials safely.
- Design and implement a relational database to store and query data.
- Build a well-documented, reusable Python class that serves as a general data analysis tool for a chosen economic topic.
- Produce a well-scoped and documented analysis.

Tasks

1. Your research question

Explore the FRED database and identify an economic question that genuinely interests you. This could be anything from "How have different sectors contributed to US GDP growth over the last decade?" to "What are the leading indicators of inflation?" or "How do unemployment rates correlate with consumer spending across US states?" FRED should be your main source of data. You may integrate additional data sources alongside it, but FRED must remain the core.

In general, remain simple in your research question. The goal of the course is to program, not to do complex economic research. A simple question that you can answer well with clean code and a clear analysis is much better than an ambitious question that leads to messy code and a vague analysis.

2. Repository and Environment Setup

Accept the GitHub Classroom invitation to create your team repository. The repository will include a basic structure for your project. All team members must contribute through it following the workflow covered in the course:

- Use branches for features or tasks. Remember the rule: **Do not push directly to `main`.**
- Write meaningful commit messages.
- Open pull requests to merge branches back into `main` (bonus).

Set up the Python environment using `uv`. The repository must include a `pyproject.toml` and a `uv.lock` so that anyone can reproduce the environment with `uv sync`. Configure `.gitignore` to exclude what needs to be excluded. Include a `README.md` that explains how to install the environment and run the code.

Note: GitHub Classroom provides a private repository for your team. It works exactly like a normal GitHub repository, but only your team members and the instructors have access to it. You can use all GitHub features (branches, pull requests, issues, etc.) as you would in any normal repository.

Your repository should follow this structure (showing committed files only):

```
project-repo/
├── .gitignore
├── README.md
├── pyproject.toml
├── uv.lock
│
├── data/
│   ├── build_fred.py     ← script to retrieve data and build the database
│   ├── er_diagram.png    ← your ER diagram
│   └── fred.db           ← generated database, committed*
│
├── src/
│   └── your_class.py     ← your Python class
│
└── notebooks/
    ├── analysis.qmd      ← analysis handout
    ├── analysis.html     ← rendered version
    ├── user_guide.qmd    ← user guide
    └── user_guide.html   ← rendered version
```

** Usually you would not commit a generated database, but for the sake of this project we ask you to do so to ensure reproducibility without needing to run the data retrieval step. In a real project, you would typically include a script to build the database and add the generated file to `.gitignore` instead.*

3. Data Collection and Database Design

3.1 Retrieve data from FRED

Retrieve time series data from the FRED API. A model script (`build_fred.py`) is provided to get you started. You need a free API key from `fred.stlouisfed.org`. Store your API key in a `.env` file and load it at runtime. Do not handcode it in your source files.

3.2 ETL, Design and implement the database

Draw an Entity-Relationship (ER) diagram for your database schema before implementing it. The diagram can be produced with Mermaid or any drawing tool; save it in the repository. Your choice for the format of the diagram does not matter: you may take a picture of a handwritten diagram, a png, jpeg, or mermaid, or else.

Conduct the ETL process (Extract, Transform, Load). Implement the schema in DuckDB. Define tables with appropriate column types, primary keys, and relationships between tables. Populate the database from the data you retrieved. For this step, you may run a single python script a single time to create the database and populate it. Once the database is built, it should be committed to the repository and no further scripts should read from the API or write to the database.

Once the database is built, all subsequent data access must be done through queries to the database, and not from the original API or intermediate files like `csvs`.

4. Data Analysis

4.1 The Python Class

You must write a Python module containing a custom class designed to perform your analysis. Every step, from reading data out of the database to generating your final figures, must be built as a method within this class. Your class should be designed not just to answer your specific question, but to serve as a versatile toolkit that someone else could use to explore related questions within your topic.

- **Structure:** Your class must cover three types of functionality: data access (reading and preparing data from your DuckDB database), helper methods (any additional cleaning, filtering, merging, or intermediate calculations needed to support your output methods), and output methods — the core tools producing tangible results such as figures, summary statistics tables, or computed metrics. You must implement **a minimum of 5 distinct, meaningful output methods**[1].
- **Output methods:** from specific to flexible. When building your output methods, we recommend a two-step approach. First, write the method to produce the output for your specific analysis. Once that works, ask yourself: how could a user adapt this to their own question? Think about what arguments would make the method more broadly useful, for instance, allowing users to filter by different criteria, specify a date range, or toggle annotations on or off.
- **Robustness and usability:** Your methods should behave predictably and give users clear feedback when something goes wrong. Think about the mistakes a user might plausibly make - passing a value that doesn't exist in the data, providing a date range where the end comes before the start, or supplying the wrong type - and raise informative errors that explain what went wrong and what to do instead. This applies to the class as a whole, not just individual methods.
- **Code style.** Every method (and the class itself) must have a docstring describing what it does, the arguments it accepts (including valid values or types where relevant), and the output it returns. The code should be logically organized and readable.

4.2 The Analysis Handout

Write a Quarto (`.qmd`) notebook laying out your economic question and key findings. Structure it as a self-contained report that walks the reader through your question and findings.

It must include:

- A brief introduction to your economic question and the data you use (including the ER diagram).
- Clean, well-labeled figures and summary tables generated by your class.
- A short discussion and conclusion of your findings.

This notebook is intended to be a short brief: think of a handout you would bring to a meeting. Introduce the question and discuss the key takeaways concisely, not exhaustively. Aim for at least 3 figures, tables, or computed metrics, but keep the document focused and to the point.

[1] Rather than aiming for more than 5 meaningful output methods, invest that effort into making your main methods flexible, user-friendly, and robust, that is what we reward.

Note: Please render this notebook to HTML before submission so that the brief can be read as a standalone document, without needing to run any code. A template is provided in the repository

4.3 The User Guide

Write a separate Quarto (`.qmd`) notebook acting as documentation for your class, bridging the gap between you as the developer and a potential end user.

It must include:

- Class overview: A brief description of the class and the economic topic it covers.
- Getting started: Instructions for any setup required (such as handling the FRED API key) and a working example showing how to instantiate the class.
- Method documentation: A clear explanation of what each method does, the arguments it accepts, and the output it returns.
- Example code: For your most important methods, provide short, isolated code examples showing how to use them and what output to expect. If a method requires prior steps in the pipeline, include those in the example.

Note: Please render this notebook to HTML before submission so that the User Guide can be read as a standalone document, without needing to run any code. A template is provided in the repository.

5. Questions and Support

Two Q&A sessions will be held during the usual exercise sessions to answer questions and provide support. The dates are the following: **8.05.2026** and **15.05.2026**. Attendance is not mandatory, but it is highly recommended to get guidance and clarify any doubts you may have.

If you have any questions about the project, only questions asked on the Canvas forum will be answered. This is to ensure that everyone has access to the same information and can benefit from the answers provided. Please check the forum regularly for updates and answers to your questions.

6. Submission

The deadline for submission is **20.05.2026 at 23:59 CET**. Late submissions will be penalized. Only submissions made through Github classroom will be accepted.

7. Grading

The project is graded across three areas. The table below shows the approximate share of points per area.

Area	Weight
A. Repository, collaboration and environment	**20%**
B. Data collection and database design	**20%**
C. Analysis and code quality	**60%**
C.1 Python class	40%-50%
C.2 Analysis notebook	5%-10%
C.3 User guide	5%-10%

A. Repository, Collaboration & Environment — 20%

All work must go through version control and GitHub classrooms following the standards presented in class. Branches must be used for all work, nothing should be pushed directly to `main`. At the time of submission, the repository must be clean and free of merge conflicts.

We expect a consistent commit history that reflects the real development of the project: commits distributed over time, with meaningful messages. Every team member must appear in the commit history.

The environment must be managed with `uv` and reproducible from a clean clone and `uv sync`. Only the right files should be committed, following the principles presented in class.

Bonus: additional points for using pull requests with reviews.

B. Data Collection & Database Design — 20%

Data must be retrieved from the FRED API. To access the FRED API, a key must be requested. The API key must be dealt with following the principles presented in the lecture.

Before implementing, an ER diagram must be drawn and integrated into the Analysis Notebook (as an image or Mermaid code or else).

The DuckDB schema must reflect that diagram: well-typed tables, primary keys, and explicit relationships. Once the database is built, it becomes the only data source for the rest of the project: The data access method(s) in the Python class must read from the database and not from any intermediate files or the original API.

Students have retrieved and stored a minimum of two different time series for at least two different variables. The minimum of two series is a starting point. Going further with more series, state-level data, additional metadata is rewarded, provided it serves the research question. Adding data and increasing complexity for its own sake is not rewarded[2].

C. Analysis and Code Quality — 60%

[2] A broader database is rewarded when it serves a clear purpose defined by the research question. Adding flexibility to your output methods - for instance, letting users filter by a characteristic and produce a figure - will naturally require more series in your database. What we do not reward is adding data that is never actually meant to be used.

C.1 Python Class — (40-50%)

We will test your class as a whole and your output methods in particular - that is, the methods that produce figures, tables, or computed metrics. Concretely, we assess the following:

- Usability of the class as a whole: The class should be intuitive to use. Think about the pipeline a user has to run to get from instantiating the class to producing output — it should be logical and as simple as possible.
- Robustness of the class as a whole: The class should anticipate mistakes users might make when setting it up or running the pipeline, and respond with clear, informative error messages.
- Correctness: When called as intended, your output methods run without error and produce the expected result[3].
- Flexibility: Your main output methods offer some degree of customization beyond your specific analysis. Helper or preparation methods that only serve to support other methods internally do not need this, but your primary outputs should.
- Input validation and error handling: Your methods anticipate plausible user mistakes and respond with clear, informative error messages that explain what went wrong.
- Docstrings: Every method (and the class itself) has a proper docstring covering its purpose, parameters, and return value.
- Code quality: The code is logically organized, easy to follow, and avoids unnecessary complexity.

C.2 Analysis Notebook — (5%-10%)

The notebook must present a clear, focused brief: a motivated question, results generated by the Python class, and a concise discussion of findings. All outputs must be professional and well-labeled. At least 3 distinct figures or tables are required. We value clarity and relevance over complexity; a simple question answered well is better than an ambitious one answered vaguely.

C.3 User Guide — (5%-10%)

We assess whether the guide is complete and clear enough that someone unfamiliar with your code could pick it up and use the class independently. It should explain how to instantiate the class, cover all methods with their arguments and return values, include working code examples for the main output methods, and be well-structured enough that a user can find what they need without reading it cover to cover.

[3] We test the methods with default values and optional values that are allowed according to your User Guide.